# Lab 31 — Risk Register, CVSS Scoring, and Quantitative Risk

You will build a CSV risk register, score each finding with CVSS v3.1, and calculate ALE (Annualised Loss Expectancy) for the top item.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

**Required software (free):** - `python3` (pre-installed) - a browser for the FIRST CVSS calculator

---

## Step 1 — Create the register

```
mkdir -p ~/grc && cd ~/grc
```

```
cat > risk-register.csv <<'EOF'
```

```
id,asset,threat,vuln,likelihood,impact,owner,strategy,status
```

```
R-001,web-prod,SQL injection,unsanitised
input,High,High,webops,Mitigate,Open
```

```
R-002,db-prod,Ransomware,no off-site
backup,Medium,Critical,dbops,Mitigate,Open
```

```
R-003,laptop-fleet,Theft,no FDE,High,Medium,it,Mitigate,In-progress
R-004,paypal-spoof,Phishing,no DMARC reject,High,High,security,Mitigate,Open
R-005,legacy-app,Vendor EOL,no patches,Low,High,procurement,Transfer,Accepted
EOF

column -t -s, risk-register.csv
```

Columns map to Security+ 5.2: identification, likelihood, impact, owner, strategy.

---

## Step 2 — Score R-001 (SQL injection) on the FIRST.org CVSS calculator

Open https://www.first.org/cvss/calculator/3.1 and enter:

| | Value |
|---|---|
| A V | Networ k |
| A C | Low |
| P | None |

| | Value |
|---|---|
| R | |
| UI | None |
| S | Unchanged |
| C / I / A | High / High / High |

Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` Base score: **9.8 (Critical)**.

---

## Step 3 — Quantitative ALE for R-001

Variables: - **Asset Value (AV)** = USD 200 000 - **Exposure Factor (EF)** = 0.6 (60% loss if exploited) - **SLE = AV × EF** = 120 000 - **ARO** (Annual Rate of Occurrence) = 0.5 (once every 2 years) - **ALE = SLE × ARO** = 60 000 / year

Compute it:

```
python3 - <<'EOF'
AV, EF, ARO = 200000, 0.6, 0.5
```

```
SLE = AV * EF




ALE = SLE * ARO




print(f"SLE = {SLE:,.0f}")




print(f"ALE = {ALE:,.0f} per year")




print(f"Spend up to {ALE:,.0f} on a control to break even.")




EOF
```

This is the budget conversation security teams have with finance.

---

## Step 4 — Decide a strategy (5.2 risk treatment)

| | When | Example |
|---|---|---|
| Mitig | Cost of control < ALE | Add WAF (Lab 16) — cost 5 000 |

| | When | Example |
|---|---|---|
| Mitigate | | |
| Transfer | Insurable, residual remains | Cyber insurance policy |
| Avoid | Risk > business benefit | Decommission the legacy feature |
| Accept | Cost of control > ALE | Sign-off by risk owner |

For R-001: ALE 60 000 vs. WAF 5 000 → **Mitigate**.

---

## Step 5 — Add Key Risk Indicators (KRIs)

```
cat > kri.csv <<'EOF'
risk_id,indicator,threshold,current,status
R-001,WAF block rate,>0,124/day,Green
R-002,Days since last off-site backup,<7,2,Green
R-003,Laptops without FDE,<5%,12%,Red
R-004,DMARC policy,reject,quarantine,Amber
EOF

column -t -s, kri.csv
```

---

## Step 6 — Risk reporting (RTBH summary)

Produce a one-page board summary:

```
python3 - <<'EOF' > board-report.md

print("# Quarterly Risk Report")

print("- Total open risks: 4")

print("- Critical: 1 (SQLi)")

print("- High: 2 (Phishing, Laptop FDE)")

print("- Medium: 1 (Ransomware backup gap)")

print("- ALE exposure (top 4): SGD 240,000")

print("- Recommended spend: SGD 35,000")

EOF

cat board-report.md
```

---

## Free online tools

- **FIRST CVSS v3.1 calculator** — https://www.first.org/cvss/calculator/3.1
- **FIRST CVSS v4.0 calculator** — https://www.first.org/cvss/calculator/4.0
- **NIST SP 800-30 Risk Assessment** — https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final
- **OpenFAIR risk taxonomy** — https://www.opengroup.org/openfair
- **Eramba (open GRC)** — https://www.eramba.org
- **SimpleRisk (open source)** — https://www.simplerisk.com

---

## What you learned

- The fields a usable risk register needs.
- Where SLE, ARO, ALE come from and how they justify spend.
- The four risk treatments and when each applies.